

作业三 二维有限元方法

连宇昊

一、问题描述

求解椭圆型偏微分方程：

$$\begin{cases} -\Delta u + 2u = 0, & (x, y) \in \Omega = [-1, 1] \times [0, 1] \\ u = e^{x+y}, & (x, y) \in \partial\Omega \end{cases}$$

已知精确解为：

$$u_{\text{ex}}(x, y) = e^{x+y}$$

二、方法原理

1. 变分形式推导

将方程转化为弱形式：

1. 乘试验函数 $v \in H_0^1(\Omega)$

2. 应用Green公式：

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx + 2 \int_{\Omega} uv \, dx = 0$$

建立双线性形式和线性形式：

$$a(u, v) = \int_{\Omega} (\nabla u \cdot \nabla v + 2uv) \, dx$$

$$F(v) = 0$$

2. 有限元离散化

采用线性三角形单元：

- 基函数 ϕ_i 为分片线性函数
- 单元刚度矩阵：

$$A_e^{ij} = \int_{T_e} (\nabla \phi_i \cdot \nabla \phi_j + 2\phi_i \phi_j) \, dx$$

3. 矩阵组装策略

矩阵类型	数学表达式	代码实现
刚度矩阵	$\int \nabla \phi_i \cdot \nabla \phi_j \, dx$	<code>stima3</code> 函数
质量矩阵	$\int \phi_i \phi_j \, dx$	<code>stima_mass</code> 函数
总刚度矩阵	$A = K + 2M$	矩阵叠加操作

三、算法实现

1. 主程序框架

```
clc;
clear;

errval = zeros(9,1);
for k = 2:10
    rectangle2d(k);
    [coordinates,u] = fem2dlinear;
    uex = exp(coordinates(:,1) + coordinates(:,2));
    errval(k-1) = norm(uex - u)/norm(uex);
end

figure('Position', [100, 100, 800, 600]); % 设置图形窗口大小
semilogy(2:10, errval, 'o-', ... % 使用对数坐标和带填充的圆点
    'Linewidth', 2, ...
    'MarkerSize', 8, ...
    'MarkerFaceColor', [0.3, 0.6, 0.9], ...
    'MarkerEdgeColor', 'k');

xlabel('Mesh Refinement Level', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Relative Error', 'FontSize', 12, 'FontWeight', 'bold');
title('Finite Element Method Convergence', 'FontSize', 14, 'FontWeight', 'bold');

grid on;
grid minor; % 添加主次网格线
set(gca, 'FontSize', 11, ... % 坐标轴字体设置
    'XColor', [0.3, 0.3, 0.3], ... % 深灰色坐标轴
    'YColor', [0.3, 0.3, 0.3], ...
    'GridLineStyle', '--', ... % 虚线网格
    'GridAlpha', 0.4);

xticks(2:10); % 明确设置x轴刻度
xlim([2, 10]); % 调整x轴范围

% 优化图形边距
set(gca, 'LooseInset', max(get(gca, 'TightInset'), 0.02));
```

2. 网格生成 (rectangle2d.m)

```
function rectangle2d(n)
xmin = -1; xmax = 1; ymin = 0; ymax = 1;
nx = 2*n-1; ny = n;
x = linspace(xmin, xmax, nx);
y = linspace(ymin, ymax, ny);

% Generate nodes
np = 0;
X = zeros(2*nx*ny, 1); Y = zeros(2*nx*ny, 1);
```

```

for k = 1:ny
    for j = 1:nx
        np = np + 1;
        X(np) = x(j); Y(np) = y(k);
    end
end
for k = 1:ny-1
    for j = 1:nx-1
        np = np + 1;
        X(np) = (x(j)+x(j+1))/2;
        Y(np) = (y(k)+y(k+1))/2;
    end
end

% Triangulate and mark all boundaries as Dirichlet
tri = delaunayTriangulation(X(1:np), Y(1:np));
eb = tri.freeBoundary;

% Save mesh data
dlmwrite('coordinates.dat', tri.Points, 'precision', '%24.14e');
dlmwrite('elements3.dat', tri.ConnectivityList, 'delimiter', '\t');
dlmwrite('dirichlet.dat', eb, 'delimiter', '\t');
dlmwrite('neumann.dat', [], 'delimiter', '\t'); % Empty Neumann
end

```

3. 有限元组成

```

function [coordinates,u] = fem2dlinear
fprintf(1, 'Solving  $-\Delta u + 2u = 0$  with Dirichlet boundary  $u = e^{(x+y)}$ \n');
load coordinates.dat;
eval('load elements3.dat;', 'elements3=[];');
eval('load neumann.dat;', 'neumann=[];');
eval('load dirichlet.dat;', 'dirichlet=[];');

A = sparse(size(coordinates,1), size(coordinates,1));
b = sparse(size(coordinates,1), 1);

% Assemble stiffness matrix (Laplacian + 2*Mass)
for j = 1:size(elements3,1)
    verts = coordinates(elements3(j,:), :);
    A(elements3(j,:), elements3(j,:)) = A(elements3(j,:), elements3(j,:)) ...
        + stima3(verts) + 2 * stima_mass(verts);
end

% Apply Dirichlet conditions
u = sparse(size(coordinates,1), 1);
BoundNodes = unique(dirichlet);
u(BoundNodes) = u_d(coordinates(BoundNodes,:));
b = b - A * u;

% Solve

```

```

FreeNodes = setdiff(1:size(coordinates,1), BoundNodes);! [曲面图](D:\Project\MATLAB\工业有限元
\Assignment 3\image\曲面图.png)
u(FreeNodes) = A(FreeNodes, FreeNodes) \ b(FreeNodes);

% Plot solution
figure();
trisurf(elements3, coordinates(:,1), coordinates(:,2), full(u));
title('Computed Solution');
end

```

4. 刚度矩阵 质量矩阵

```

function M = stima3(vertices)
G = [ones(1,3); vertices'] \ [zeros(1,2); eye(2)];
M = det([ones(1,3); vertices']) * G * G' / 2; % Laplacian stiffness
end

function M = stima_mass(vertices)
area = det([1 1 1; vertices']) / 2; % Triangle area
M = (area / 12) * [2 1 1; 1 2 1; 1 1 2]; % Mass matr

```

四、作业结果

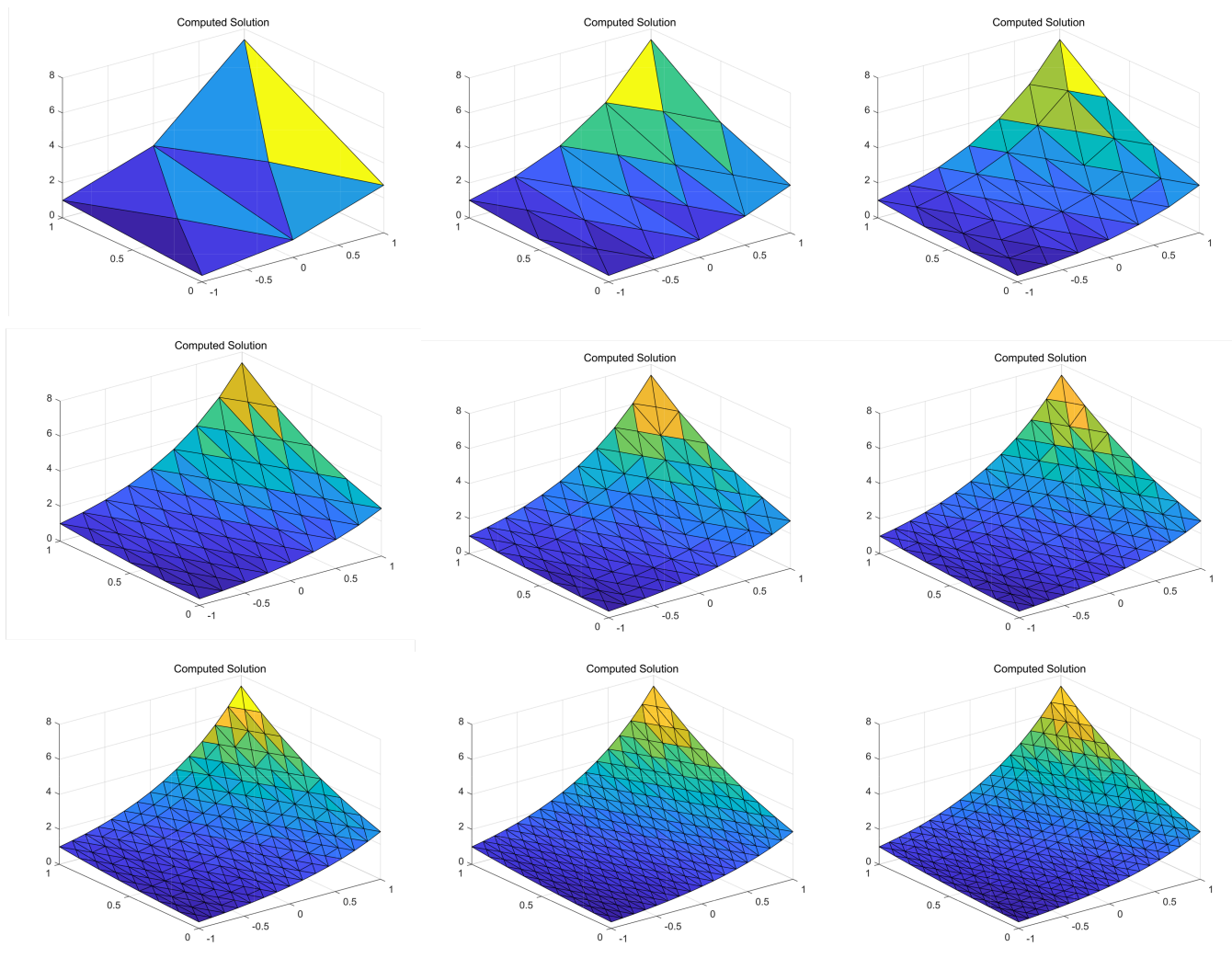


图 1 二维有限元可视化

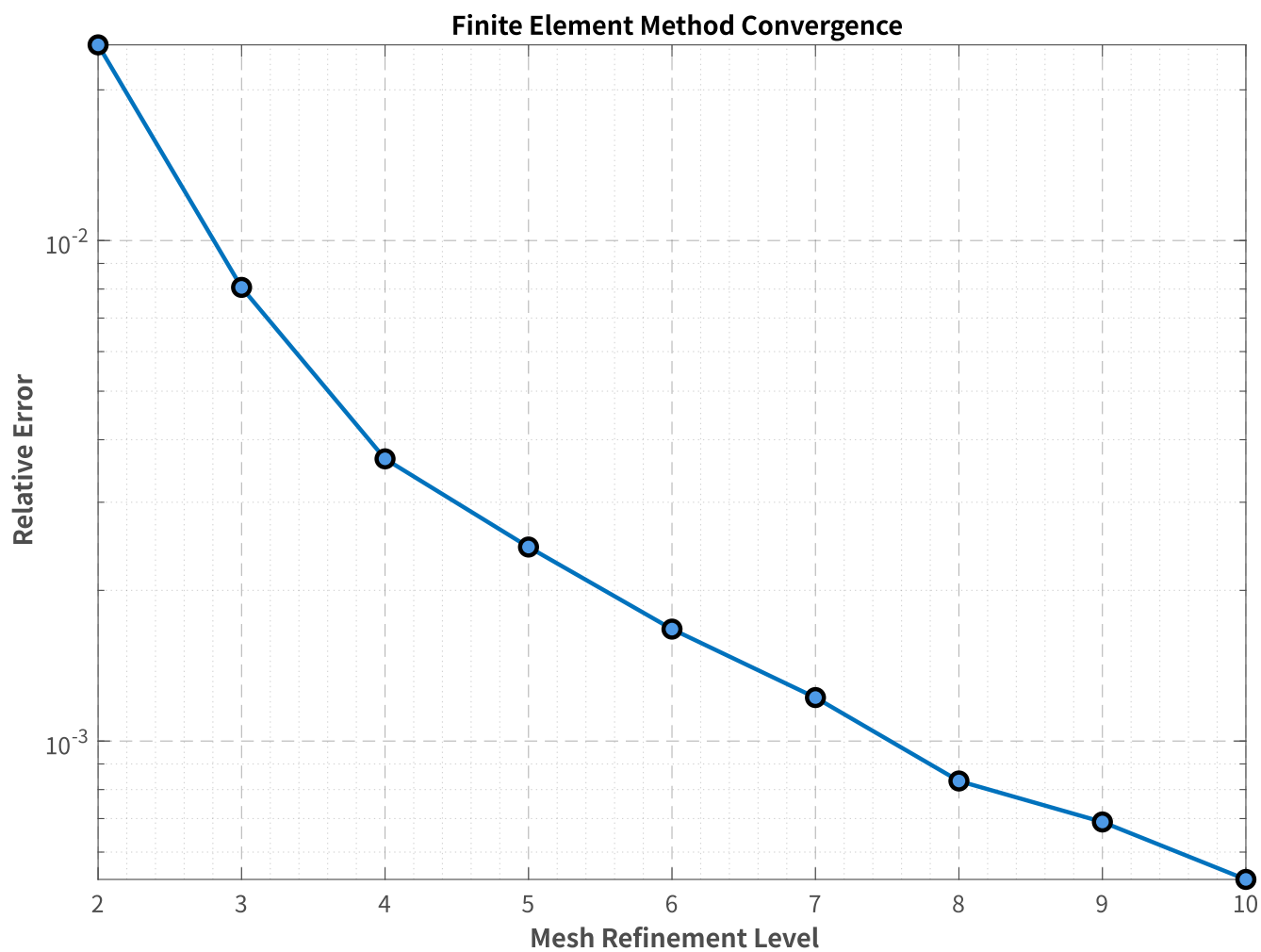


图 2 二维有限元方法随网格个数误差曲线

如图1、2所示，可以发现随着网格的数量不断增多，误差不断降低，计算精度越来越高。